

Session 4B — Homework

Built-in array methods: replacing loops you already know how to write

Hand in before: the agreed date at the end of the course **Time you need:** about 30–40 minutes for CORE

Objective

You've written `for...of` loops all course — walking every item in an array, checking something, and deciding what to do. Tonight you'll learn that JavaScript arrays already have built-in tools for three very common versions of that pattern, and you'll use them on TaskFlow code that already has exactly that shape.

No new feature. Same app, same behavior. All five compiler rules from class stay `true` all night. Every homework solution must compile with **zero errors**.

Use normal function callbacks, not arrow functions — the same `function (x) { ... }` shape you've already used for every event listener this course.

```
users.find(function (user) {  
    ...  
});
```

not

```
users.find((user) => ...);
```

What you already have

Start from your own Session 4B End file, or `05-session-end/project/` if you need a clean copy. Confirm it compiles with **zero errors** and runs exactly like class before you start.

CORE — everybody does this — `find()`

The idea

Look at `getUserName()`. Its job is: search through `users`, and stop as soon as one matches — it is looking for exactly **one** item.

```
loop
→ check each user
→ stop when one matches
```

JavaScript arrays have a built-in method that describes exactly this: `.find()`.

A tiny example, before `TaskFlow`:

```
const numbers = [3, 7, 12, 18];

const firstBig = numbers.find(function (n) {
  return n > 10;
});

// firstBig is 12 – the first item where the callback returned true
```

`find()` :

- searches the array,
- calls your function once per item until one returns `true`,
- returns that **first matching item**,
- or returns `undefined` if nothing ever matched.

Your task

Rewrite `getUserName()` to use `.find()` instead of the `for...of` loop.

Recipe:

1. Call `users.find(...)` with a normal function callback that receives one `user` and returns `true` when `user.id === userId`.
2. Store the result in a `const` (its type will be `User | undefined` - TypeScript already knows this, because `strictNullChecks` is on and `find()` might not find anything).

3. Because the result might be `undefined`, check it before reading `.name` - use the same `if (x)` idea you already use for `if (!x) { return; }`, just checking for the opposite case this time.
4. If nothing was found, keep the existing `"Unknown person"` fallback.

Do not use optional chaining (`?.`). A plain `if` check is all this needs.

CORE acceptance criteria

- ☐ `getUserNames()` uses `.find()` instead of a `for...of` loop.
- ☐ A normal function callback is used, not an arrow function.
- ☐ The `"Unknown person"` fallback still works when nobody matches.
- ☐ `tsc --project tsconfig.json` compiles with **zero errors**, all five rules still `true`.
- ☐ Owner names still appear correctly under every task title.

STRETCH — optional — `filter()`

The idea

Look at `getVisibleTasks()`. Its shape is:

```
create empty visibleTasks
↓
loop tasks
↓
calculate matchesFilter, matchesSearch, matchesPerson
↓
if all true
↓
visibleTasks.push(task)
↓
return visibleTasks
```

This is exactly what `.filter()` does: it keeps the items for which your callback returns `true`, and returns a new array of just those items.

Your task

Replace only the **empty array + loop + push + return** structure with `.filter()`. Keep every existing condition exactly as it is — `matchesFilter`, `matchesSearch`, and `matchesPerson` are still calculated the same way, inside the callback.

Recipe:

1. Remove `const visibleTasks = [];` and the final `return visibleTasks;`.
2. Call `tasks.filter(...)` with a normal function callback that receives one `task`.
3. Move the existing filter/search/person logic inside that callback, unchanged.
4. Instead of `visibleTasks.push(task)`, `return` the combined boolean directly: `return matchesFilter && matchesSearch && matchesPerson;`
5. `return` the result of `tasks.filter(...)` directly from `getVisibleTasks()`.

Keep `search = searchText.toLowerCase()` calculated once, before `.filter()` starts - moving to `.filter()` doesn't change that reasoning at all.

STRETCH acceptance criteria

- ☐ `getVisibleTasks()` uses `.filter()` instead of an empty array + loop + push.
- ☐ Every existing condition (`matchesFilter`, `matchesSearch`, `matchesPerson`) is unchanged.
- ☐ A normal function callback is used, not an arrow function.
- ☐ `tsc --project tsconfig.json` compiles with zero errors.
- ☐ Status filters, search, person selection, and all combinations of the three still work exactly as before.

CHALLENGE — optional — `reduce()`

The idea

Look at `updateTaskSummary()`. The completed count is built like this:

```
let completed = 0;

for (const task of tasks) {
  if (task.completed) {
```

```
        completed++;  
    }  
}
```

This is a **running total** built up one task at a time — exactly what `.reduce()` is for.

Your task

Replace **only** the completed-count loop with `.reduce()`. Keep `const pending = tasks.length - completed;` and every DOM update line exactly as they are.

Recipe:

```
const completed = tasks.reduce(function (count, task) {  
    if (task.completed) {  
        return count + 1;  
    }  
  
    return count;  
}, 0);
```

Walk through it slowly:

- `0` — the starting count, before any task has been looked at.
- `count` — the running total, carried from one task to the next.
- `task` — the current task being looked at.
- `return count + 1` — when this task is completed, the new running total is one more than before.
- `return count` — otherwise, the running total doesn't change.
- **The final value `.reduce()` hands back** — the running total after every task has been visited — becomes `completed`.

Do not shorten this to a ternary. Two explicit `if / return` lines are the required shape.

CHALLENGE acceptance criteria

- ☐ The completed-count loop inside `updateTaskSummary()` is replaced with `.reduce()`, starting at `0`.
- ☐ `const pending = tasks.length - completed;` is unchanged.
- ☐ A normal function callback is used, not an arrow function.
- ☐ `tsc --project tsconfig.json` compiles with zero errors.

- ☐ Stats and the progress sentence still show the correct numbers.